

Functions

Functions

- Until now, to repeat a segment of code, we would be required to copy and paste again
 - error prone
 - messy
 - redundant
- There is a way around this

User Defined Function

- we have used built in functions and methods that come with python
- A programmer can make custom functions to suit their needs

Function Basics

- A *function* is a named series of statements
 - A *function definition* consists of
 - function name
 - indented block of statements
 - A *function call* is an invocation of the function's name, causing the function's statements to execute

Function Basics

- *def* is a keyword that is used to create new functions
- A function can only return one item
 - list or tuple with multiple elements can be returned
 - We can also pack multiple items into a tuple
 - if no return is used, it returns **None**

Parameters

- A programmer can influence a functions behavior via input
 - A *parameter* is a function input specified in a function definition
 - An *argument* is a value provided to a function's parameter during a function call

Parameters

- May have no parameters or multiple parameters
 - each parameter is separated with a comma

Printing from a Function

- You may print from a function
 - typically will not have a return value
 - this is called a *void function*

Dynamic Typing

- Consider the following function:

```
def add(x, y):  
    return x + y
```

Dynamic Typing

- *Polymorphism* - when adding different types using the above function is known as polymorphism
- Python uses dynamic typing which means the data type is determined as the program executes
- Other languages use *static typing* in which it requires the programmer to define the type of every variable and every function parameter in a program's source code

Why Functions

- readability
- ease of use
- modular development
- reduces redundancy